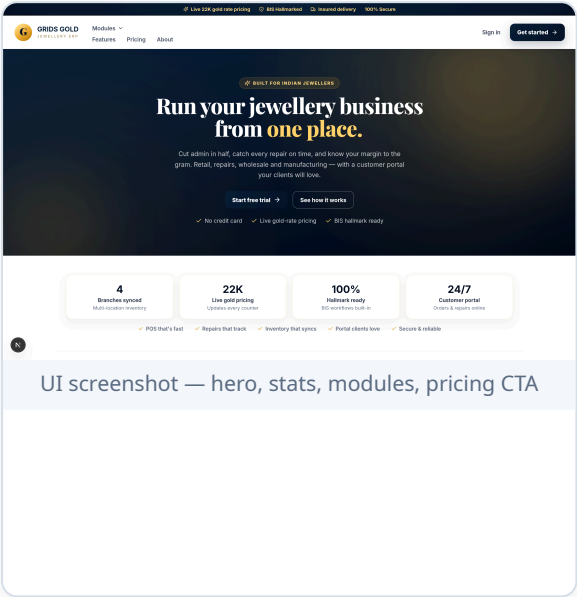


GET /

Marketing Homepage

app/page.tsx · LandingNav · ScrollReveal · TypewriterText · landingModules.ts



```
app/page.tsx — structure & routing

"use client"; // animations + interactive nav

import { moduleHighlights } from
"./lib/landingModules";

const stats = [
  { value: "4", label: "Branches synced" },
  { value: "22K", label: "Live gold pricing" },
];

export default function LandingPage() {
  return (
    <section className="landing-hero">
      <TypewriterText
        phrases=["one place.", "every branch."] />
      <Link href="/login?signup=1">Start free
trial</Link>
    </section>
  );
}
```

```
app/lib/landingModules.ts — SRS module map → real routes

export const moduleGroups = [
  { label: "Daily operations", items: [
    { title: "POS & Sales", href: "/pos" },
    { title: "Repairs", href: "/repairs" },
  ]},
];
// flatMap → homepage grid + LandingNav mega-menu
```

EXPLAIN THIS PAGE

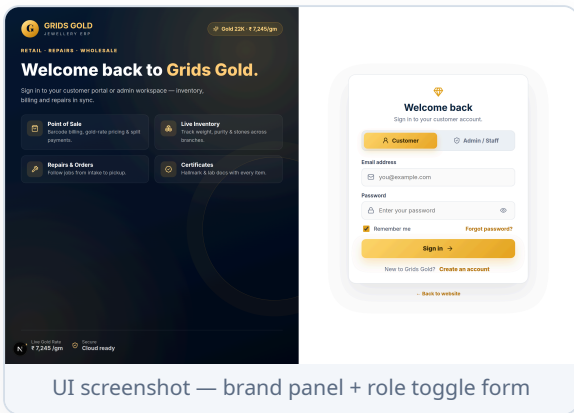
- **Next.js App Router:** file app/page.tsx = route /
- **Client Component** — uses hooks, Intersection Observer, typewriter timers
- **Data-as-config** — stats/features/plans are arrays → .map() to JSX
- **Static prerender** — no API on load; Vercel-friendly
- **next/link** — client navigation to /login without full reload

Say: “The marketing layer is decoupled from the ERP. All CTAs funnel users into the auth entry at /login.”

GET /login · /login?signup=1

Login & Sign Up

app/login/page.tsx · app/components/AuthPanel.tsx · app/lib/store.tsx



app/login/page.tsx — URL drives signup mode

```
const params = useSearchParams();
const isSignup = params.get("signup") === "1";

return (
  <Suspense fallback={<Loading />}>
    <AuthPanel
      initialMode={isSignup ? "signup" : "signin"}
    />
  </Suspense>
);
```

app/components/AuthPanel.tsx — submit → store → redirect

```
async function handleSubmit(e) {
  e.preventDefault();
  if (isAdmin) {
    signup({ role: "admin" });
    router.push("/dashboard"); return;
  }
  if (isSignup) signup({ name, email, role });
  else login(email, role);
  router.push(role === "admin" ? "/dashboard" : "/portal");
}
```

```
// Optional: supabase.auth.signUp / signInWithPassword
// when NEXT_PUBLIC_SUPABASE_* env vars are set
```

app/lib/store.tsx — temporary auth state (SRS Phase 1 demo)

```
const signup = useCallback((u) => {
  setCurrentUser({ name: u.name, email: u.email, role: u.role });
}, []);

const logout = useCallback(() => setCurrentUser(null), []);
// Persisted to localStorage key gg_state_v4
```

EXPLAIN THIS PAGE

- **Two roles:** Customer → /portal · Admin → /dashboard
- **Controlled form** — React state owns every input field
- **Context API** — useStore() shares user across all routes
- **Sign out** clears currentUser → redirects to /login
- **SRS enums** in srs.ts ready for real DB columns

Say: “Auth UI is production-ready; persistence is mock. Supabase Auth + Postgres is the planned SRS backend.”

Phase 1 SRS · Deep Dive v2

What Is Done vs What Needs To Be Done

Aligned to SRS requirement IDs referenced in the codebase (app/lib/srs.ts, module pages, app/data.ts screen map).

SRS area	Requirement IDs	Frontend today	Still needed (per SRS)
Platform & Auth	Access, roles	Partial Login UI, role toggle, mock session	Supabase Auth, RBAC (/roles), branch gate (/branch-selection), route guards, password reset
Organization	FR-ORG-002	Partial Branch switcher in top bar	Branch/location master, showroom/safe/tray/warehouse locations in DB
CRM	FR-032, FR-CRM-001/002	Done UI Customer list + add form	Profile, purchase history, follow-ups, consent audit trail in Postgres
Jewelry / Items	FR-051 – FR-066	Done UI Catalog, live-rate pricing formula	Product detail page, certificates/docs, hallmark workflow
Inventory	FR-067 – FR-082, FR-INV-001	Done UI Stock, transfers, adjustments, cycle count	Barcode tag printing, movement_type enums in DB, location-level stock
POS & Sales	FR-RET-001, counter	Done UI POS checkout → invoice + stock	Returns/exchange, quotations, layaway, split-payment ledger
Repairs	FR-REP-001, §19.3 statuses	Done UI Intake, status board, deposits	Full 7-stage workflow, ticket detail, custom orders, pickup/delivery
Purchasing	FR-045, FR-PUR-002	Done UI Suppliers, PO list	Goods received (GRN), supplier payments, PO approval chain
Manufacturing	FR-185 – FR-191	Partial Work orders dashboard	Karigar tracking, material issue, finished goods receipt, wastage
Finance	FR-143 – FR-152, FR-FIN-003	Partial Overview, expenses, aging tables	Payments, cash/bank ledger, GST module, real AR/AP posting
Reports	FR-RPT-001, Deep Dive §12	Done UI 12 reports in 5 families + CSV	Server-side aggregates, scheduled reports, PDF email
Wholesale	B2B module	Partial Wholesale screen	Bulk orders, dispatch challan, dealer pricing rules
Customer portal	Online channel	Done UI Catalog, wishlist, orders, repairs	Payments, notifications (SMS/WhatsApp), real order fulfilment
Settings & audit	System	Partial Settings, gold rates	Invoice print settings, masters, import/export, audit log
Database	DDL / Postgres	Not wired localStorage demo store	Supabase PostgreSQL, Storage, replace store.tsx with API layer

PLACEHOLDER SCREENS STILL ON ERPSCREEN FALLBACK

~40 routes in app/data.ts render generic placeholders via app/[...slug]/page.tsx until built as dedicated pages. Examples:

- Sales: /sales/returns, /quotations, /layaway
- Inventory: /barcode-tags · CRM: /customers/profile, /follow-ups
- Purchasing: /goods-received, /supplier-payments
- Manufacturing: /job-work-orders, /wastage-tracking
- Finance: /payments, /ledger, /gst-tax
- System: /roles, /audit-log, /import-export

SRS ENUMS ALREADY CODED (READY FOR DB)

```
app/lib/srs.ts — Deep Dive status dictionaries

INVOICE_STATUSES: draft | posted | paid | partial | voided | refunded
REPAIR_STATUSES:  received → diagnosis → awaiting_approval → in_progress → ready → delivered
PO_STATUSES:      draft | approved | partial | closed | cancelled
MOVEMENT_TYPES:   receive | sale | return | transfer_out | transfer_in | adjustment ...
SRS_REPORT_CATEGORIES: Sales | Stock | Repairs | Finance | Tax  // §12
```

RECOMMENDED NEXT STEPS (SRS ORDER)

- 1. **Supabase** — Auth + PostgreSQL schema from DDL docs
- 2. **API layer** — replace localStorage store with server calls
- 3. **Route guards** — protect /dashboard and /portal
- 4. **Promote placeholders** — returns, GRN, ledger, roles, audit log
- 5. **Production hardening** — tests, CI, Vercel env vars, monitoring

Closing line: “The frontend implements the SRS module map and core workflows as an interactive prototype. What remains is wiring the Deep Dive data model to PostgreSQL and completing the secondary workflow screens.”